ASSIGNMENT 1 – CO1

DESIGN, IMPLEMENT AND VALIDATE THE FARE LOGIC OF A CITY METRO SMART-CARD SYSTEM

Name	Nirmal Kumar T
Register Number	1972260020
Course	Object Oriented Programming with C++
Course Code	DSA0113
GitHub Repository	https://github.com/Nirmal637/DSA01-1972260020-Assignment1

TASK 1: UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

This submission presents an object-oriented analysis of the metro smart-card fare problem. Task 1 focuses on identifying the important objects, their data and behaviour, comparing procedural and object-oriented approaches, and stating the rules that every valid journey must satisfy.

1. Problem Understanding

The metro system has Inner, Middle and Outer travel zones. The base fare is determined by the farthest zone reached during a journey: Rs 10 for Inner, Rs 20 for Middle and Rs 30 for Outer. A peak-hour surcharge of 20% applies from 8:00 AM to 10:00 AM and from 5:00 PM to 8:00 PM. A passenger uses a stored-value smart card. A journey is permitted only when at least Rs 50 remains in the card balance after the fare is deducted. Student and senior passenger concessions are also mentioned, but their percentages are not specified in the problem statement.

The system must also be designed with future growth in mind. The daily number of journeys may increase from 20,000 to 500,000 and then to 3,000,000. Possible future changes include adding a fourth zone, introducing a weekend discount and supporting monthly passes.

2. Identification of Objects

The following objects represent the main entities and responsibilities in the problem.

Object	Important data (attributes)	Main behaviour (methods)	Justification
SmartCard	Card ID; balance; card status; passenger category	Check balance; deduct fare; validate minimum remaining balance; add value	Stores the value used to pay and protects balance-related rules.
Passenger	Passenger ID; name; category (regular/student/senior)	Request journey; identify passenger category; use a card	Passenger category is relevant because concessions may depend on it.
Journey	Origin zone; destination zone; entry time; exit time; calculated fare	Determine farthest zone; calculate fare; validate and complete journey	Represents one trip and the facts needed to calculate its fare.
Zone	Zone ID; zone name; fare tier or base fare	Provide zone identity and associated fare level	Makes zone-based fare rules explicit and allows a new zone to be added.
FareTable	Zone fares; peak surcharge rate; concession rules; weekend rules	Look up base fare; apply time-based and concession rules	Centralises fare policy instead of scattering values throughout the program.

3. Procedural Programming vs Object-Oriented Programming

In a procedural solution, the system could use global variables and separate functions such as calculateFare(), checkBalance() and deductFare(). Card balances, passenger categories and journey details would be passed between functions or stored in shared data structures. This can work for a small program, but related data and rules are not grouped together.

In an object-oriented solution, SmartCard, Passenger, Journey, Zone and FareTable objects keep related data and behaviour together. Each class has a clear responsibility. For example, SmartCard can ensure that a deduction never leaves the required minimum balance, while FareTable handles fare policy. This separation makes the design easier to extend and maintain.

Aspect	Procedural approach	Object-oriented approach
Organisation	Data and functions are separate; shared/global data may be used.	Data and related operations are grouped into classes and objects.
Per-journey time	O(1) for a fixed number of fare checks and calculations.	O(1) for the same fixed number of operations.
Per-card memory	O(1) for a fixed set of card fields.	O(1) for a fixed set of object fields.
Changing a fare rule	May require finding and updating several functions or shared conditions.	Usually localised in FareTable or the relevant policy class, with dependent tests updated.
Extensibility	New features can increase coupling between functions and shared data.	New zone or pass types can be modelled with additional data/classes and clear interfaces.

4. Object-Oriented Principles

Encapsulation: SmartCard keeps its balance private and exposes controlled operations such as getBalance() and deductFare(). This prevents unrelated parts of the program from changing the balance without validation.

Abstraction: A Journey can request a fare from FareTable without needing to know every internal calculation step. The fare policy is exposed through a simple operation such as calculateFare().

Inheritance: StudentPassenger and SeniorPassenger may inherit common passenger information and behaviour from Passenger if they share a stable common structure. Their concession eligibility can be specialised. Inheritance should be used only where the relationship is genuinely an “is-a” relationship.

Polymorphism: Different passenger categories can implement a common concession calculation interface. The fare system can call the same operation, while the appropriate student or senior implementation supplies its own rule. Exact concession percentages must come from the official specification.

5. Rules and Conditions for a Valid Journey

Every journey must satisfy the following conditions. These rules should be enforced by the system rather than relying only on the user interface.

1. Valid card: The smart card must exist and be active before a journey can be charged.

2. Valid zones: The origin and destination must be recognised zones in the metro network.

3. Correct base fare: The fare must be based on the farthest zone reached: Inner Rs 10, Middle Rs 20 or Outer Rs 30.

4. Peak-hour surcharge: Add 20% during 8:00 AM–10:00 AM and 5:00 PM–8:00 PM. For clarity, the boundaries are treated as start-inclusive and end-exclusive: 8:00 AM and 5:00 PM are peak times; 10:00 AM and 8:00 PM are not.

5. Concession eligibility: Apply a student or senior concession only when the passenger's category is verified and the official concession percentage and calculation order are known.

6. Minimum remaining balance: The journey may be completed only if the balance after all applicable fare adjustments and deduction is at least Rs 50.

7. Atomic deduction: If validation fails, the journey must be rejected and the card balance must remain unchanged.

8. Accurate record: A completed journey should record its zones, time, fare and card reference for audit and customer support.

6. Procedural-Style Alternative (Pseudocode)

The same fare process could be implemented procedurally using shared records and functions. The following high-level pseudocode illustrates the flow without requiring a specific programming language.

```
START
Read card, passenger category, origin, destination and journey time
IF card is invalid OR zones are invalid: reject journey
ELSE
baseFare = look up fare for farthest zone reached
IF journey time is in a peak window: baseFare = baseFare * 1.20
Apply verified concession rule, if applicable
IF cardBalance - finalFare >= 50:
deduct finalFare from cardBalance
record journey as successful
ELSE:
reject journey; do not change cardBalance
END IF
STOP
```

7. Complexity and Scale Considerations

For a fixed number of zones and fare conditions, fare lookup, time-window checking and balance validation require a constant number of operations. Therefore, the time per journey is O(1) in both procedural and object-oriented designs. The memory required for one card record/object is also O(1), assuming a fixed set of fields. Object orientation does not automatically make the algorithm faster; its main benefit here is clearer responsibility, controlled access to data and easier extension.

At millions of journeys per day, the system should additionally consider database indexing, transaction safety, concurrent card updates, logging, monitoring and scalable storage. These are system-level concerns beyond the constant-time fare calculation itself.

8. Assumptions and Points Requiring Confirmation

The assignment statement does not specify the exact student and senior concession percentages. It also does not specify whether a concession is applied before or after the peak surcharge. These values and the order of application must be confirmed from the instructor or a complete official fare policy; they must not be invented.

For this analysis, peak-hour intervals are interpreted as start-inclusive and end-exclusive. The Rs 50 minimum is checked after all fare adjustments. A card with exactly Rs 50 remaining after deduction is accepted. If the journey is rejected, no amount is deducted.

The base fare is interpreted as the fare for the farthest zone reached, as described in the problem statement. If the official specification defines a different origin-destination fare matrix, that matrix should replace this assumption.

9. Conclusion

The metro smart-card problem is well suited to an object-oriented design because it contains distinct entities with related data and responsibilities. SmartCard, Passenger, Journey, Zone and FareTable provide a clear starting model. Encapsulation protects card balance rules, abstraction separates fare calculation from journey processing, and inheritance/polymorphism can support verified passenger-specific concessions. The key validation rule is that a completed journey must leave at least Rs 50 in the card. Both procedural and object-oriented solutions can perform the basic fare calculation in O(1) time, while the object-oriented approach provides a more maintainable structure for future fare changes and system growth.

Submission note: This is an object-oriented analysis task. No executable program or fabricated execution output is included because Task 1 asks for problem understanding and object identification rather than compulsory coding.

End of Task 1